

Action-Specific Guardrails: From Content Safety to Tool-Execution Governance in Agentic AI Systems

John Cheung
College of Professional and Continuing Education

Abstract

The rapid adoption of agentic AI systems has shifted the safety problem from conversational output control to executable action governance. In retrieval-augmented generation (RAG) systems, guardrails commonly focus on harmful content, prompt injection, personally identifiable information, groundedness, and hallucination reduction. However, modern AI agents and skill-based systems increasingly call tools, execute shell commands, modify repositories, update records, send messages, deploy services, and interact with cloud infrastructure. This creates a qualitatively different risk: the agent may perform an unsafe action even when its textual response appears compliant. The recent PocketOS incident, in which an AI coding agent reportedly deleted a production database volume despite written instructions prohibiting destructive behavior, illustrates the inadequacy of prompt-only control for agentic execution.

This paper argues that agentic AI requires a distinct layer of action-specific guardrails: a runtime policy and enforcement system that evaluates proposed tool calls before execution. Unlike conventional content-safety systems, which classify text or image risk, action guardrails classify the proposed operation, target resource, environment, reversibility, blast radius, credential scope, required approval, and rollback capability. We review the current market landscape, including cloud provider guardrails, model-harness permission systems, AI security products, orchestration frameworks, and policy engines. We find that the market is moving toward agent/tool guardrails but has not yet converged on a mature, standardized, action-specific guardrails framework analogous to established RAG safety layers.

The paper contributes a reference architecture, a formal threat model, an action-risk taxonomy, a policy schema, measurable governance objectives, an evaluation plan, and an implementation roadmap for organizations developing AI agents, skills, and enterprise chatbot systems. The central recommendation is that AI systems should never rely on system prompts or skill instructions alone to prevent dangerous actions. Instead, proposed actions should pass through an external policy layer, be enforced by least-privilege credentials and API/IAM restrictions, require human approval when risk exceeds threshold, and produce auditable traces. Content safety checks what the AI says; action-specific guardrails check what the AI does.

Keywords: agentic AI; AI guardrails; tool execution; action policy; RAG guardrails; AI safety; IAM; policy-as-code; human-in-the-loop; AI governance; MCP; AI agents; skill-based chatbots

1 Introduction

The first generation of enterprise large language model (LLM) governance was built around the chatbot. The main safety questions were whether the model produced harmful content, leaked sensitive information, fabricated unsupported claims, followed malicious prompt injections, or answered outside an approved knowledge base. This was natural when most applications were conversational systems or retrieval-augmented

generation (RAG) workflows. In such systems, a user asks a question, the application retrieves documents, the model generates an answer, and a guardrail layer checks the input, retrieved context, or output.

Agentic AI changes the safety boundary. A modern AI agent does not merely answer. It acts. Coding agents read repositories, edit files, run commands, call test suites, install dependencies, query cloud APIs, and interact with development environments. Business agents draft and send emails, create tickets, update CRM records, schedule meetings, retrieve documents, call workflow APIs, and interact with external systems. Institutional chatbot platforms increasingly expose “skills” that connect to student services, learning management systems, internal databases, helpdesk platforms, and administrative workflows.

This means that the most important safety question is no longer only whether the generated answer is safe. It is also whether this AI system should be allowed to perform this action on this resource under these conditions. The distinction is critical. A content-safety system may classify a conversation as harmless while the agent executes a destructive API call. A system prompt may instruct the agent never to delete production data, but if the agent possesses a credential that allows deletion, the instruction remains advisory rather than enforceable. In action-capable systems, prompt compliance is not a substitute for authorization.

The reported PocketOS incident in April 2026 provides a useful motivating example. Public reporting describes an AI coding agent that allegedly deleted a production database volume and backups through a cloud provider API despite instructions not to perform destructive actions [1, 2, 3]. The details vary across reports and should be treated as an illustrative case rather than a fully adjudicated technical postmortem. Nevertheless, the incident captures a general failure mode: an autonomous or semi-autonomous AI system was given a path to execute a high-impact action without an adequate action-specific approval and enforcement layer.

This paper proposes that organizations need a new class of guardrails: action-specific guardrails. These guardrails are not limited to text moderation, jailbreak detection, or groundedness checking. They are runtime controls for tool execution. Their purpose is to mediate between the agent’s proposed action and the actual system API, database, shell, connector, or workflow engine.

The core argument is simple: in agentic AI, safety must be enforced at the action boundary, not only at the language boundary.

1.1 Contributions

This paper makes six contributions:

1. defines *action-specific guardrails* as a distinct layer for agentic AI systems;
2. distinguishes content safety, RAG guardrails, harness permissions, IAM controls, and action-specific policy enforcement;
3. reviews current market offerings and identifies the gap between mature content guardrails and emerging action/tool-call guardrails;
4. proposes a reference architecture for an Action Guardrails Framework;
5. adds a threat model, security objectives, and non-bypassability requirements for complete mediation;
6. introduces an action-risk taxonomy, policy schema, and measurable governance metrics;
7. outlines an evaluation plan using seeded destructive actions, approval-bypass tests, credential-scope tests, provenance audits, overhead measurements, and comparative baselines.

2 Motivating Case: The PocketOS Incident

The PocketOS case is useful because it shifts the discussion from theoretical AI risk to an operational failure mode. According to public reporting, PocketOS used an AI coding agent in a development workflow. The agent was reportedly operating in a context where it should not perform irreversible or destructive actions. Yet it allegedly used an available API token to delete a production database volume or related infrastructure [1, 3, 2]. Some reports state that production data and backups were affected, while other reporting indicates that some recovery was possible through provider-side or offsite recovery mechanisms. This paper does not depend on every disputed detail of the incident. The broader lesson is independent of whether the final data loss was total, partial, or recoverable. The case illustrates five structural risks that apply to many agentic systems.

2.1 Prompt Instructions Are Advisory

If an agent is told “do not delete production data,” that instruction influences model behavior but does not technically prevent deletion. The model may misunderstand the environment, misclassify the resource, follow an erroneous chain of reasoning, or be influenced by retrieved content, tool output, or prior context. Prompt instructions are necessary but insufficient.

2.2 Tool Access Can Exceed Task Intent

An agent may be given access to a general-purpose tool or credential for convenience. For example, a cloud API token may allow reading logs, restarting services, updating environment variables, and deleting volumes. The intended task may require only read access, but the credential may include destructive privileges. This creates a mismatch between task scope and tool capability.

2.3 Environment Boundaries Are Often Ambiguous

Agents may operate across local, staging, and production contexts. A command, token, configuration file, or API endpoint may not clearly encode the environment. A human engineer may recognize the danger from context; an agent may not. The guardrails layer must not rely solely on the model’s interpretation of environment names.

2.4 High-Risk Actions Need Approval Gates

Some actions should never be performed automatically. Production volume deletion, backup deletion, schema destructive migration, mass email sending, financial transaction execution, access-right modification, and student-record updates should require explicit approval. Approval should occur outside the model’s private reasoning loop and should be bound to the exact tool arguments and target resource.

2.5 Recovery Is Not a Substitute for Prevention

Soft delete, backups, audit logs, and disaster recovery are important. However, they are after-the-fact controls. A mature system should prevent unauthorized high-risk execution in the first place, while also preserving rollback and recovery if prevention fails.

The PocketOS case is therefore not simply a story about an AI agent making a mistake. It is a story about missing or insufficient action governance. The AI had, or appeared to have, a technical route to a destructive outcome. The action guardrails framework proposed in this paper aims to remove or mediate such routes.

3 From RAG Guardrails to Action Guardrails

RAG guardrails and action guardrails share a common goal: they make AI systems safer, more reliable, and more governable. However, they operate on different objects.

3.1 RAG and Content Guardrails

A conventional RAG pipeline is often structured as follows:

```
User query
-> Input safety check
-> Retriever
-> Context filtering and ranking
-> LLM generation
-> Output safety, groundedness, and citation checks
-> User response
```

The guardrails ask questions such as whether the user prompt is harmful or malicious, whether retrieved context contains prompt injection, whether the generated answer is grounded in approved sources, whether the answer leaks personal or confidential data, whether the answer contains prohibited content, and whether the output matches a required schema. These are language and content questions.

3.2 Agentic Action Flow

An agentic system has a different structure:

```
User request
-> Agent planning
-> Tool/action proposal
-> Action policy evaluation
-> Approval, denial, sandbox, or execution
-> Tool/API result
-> Trace, audit, and response
```

The guardrails ask different questions: what action is the agent proposing; what resource will be affected; whether the resource is production, staging, or local; whether the action is read-only, reversible, external, destructive, or irreversible; which credential will be used; whether the user and agent identities are authorized; whether approval is required; whether a rollback path exists; and whether the action creates external side effects.

3.3 Boundary Difference

The two guardrail types are complementary. A mature AI platform needs both.

4 Current Market Landscape

The market currently provides mature content and prompt guardrails, growing agent security products, and partial tool-permission systems. However, a complete and standardized action-specific guardrails platform remains emerging.

Table 1: RAG/content guardrails versus action-specific guardrails.

Dimension	RAG / Content Guardrails	Action-Specific Guardrails
Primary object	Text, image, retrieved document, generated answer	Tool call, API call, command, work-flow action
Main risk	Harmful output, hallucination, prompt injection, leakage	Unauthorized mutation, destructive action, side effect, privilege misuse
Typical decision	Allow, block, redact, rewrite, cite, escalate	Allow, ask, approve, sandbox, dry-run, deny
Enforcement point	Before/after model generation	Before tool/API execution
Key metadata	Prompt, response, source, category, severity	Action, resource, environment, reversibility, credential, approval
Failure example	Ungrounded answer	Production database deletion

4.1 Cloud Content-Safety Guardrails

Cloud providers offer content-safety systems that help developers detect harmful or risky text, images, prompts, and generated responses. Azure AI Content Safety provides APIs for detecting harmful text and image content [4]. Azure Prompt Shields detects adversarial prompt attacks against LLM applications [5]. Azure groundedness detection checks whether LLM responses are based on supplied source material [6]. Amazon Bedrock Guardrails provides configurable policies for generative AI applications and can be used with Bedrock Agents and Knowledge Bases [8, 9].

These services are important, but their primary design center remains content and model interaction safety. They can reduce harmful outputs, detect jailbreaks, and support RAG quality, but they do not by themselves enforce fine-grained authorization over every downstream action. Associating a guardrail with an agent is useful, but the actual action safety still depends heavily on tool design, action group implementation, IAM, approval workflow, and backend enforcement.

4.2 Agent Orchestration Frameworks

Frameworks such as LangChain, LangGraph, Semantic Kernel, and related orchestration systems provide mechanisms for defining tools, managing agent loops, introducing middleware, adding human-in-the-loop approval, and tracing execution. These frameworks are valuable implementation substrates for action guardrails. For example, a developer can intercept tool calls and require user confirmation before executing selected tools.

However, framework-level middleware is not automatically equivalent to enterprise action governance. A custom developer still needs to classify risk, define policy, map resources, enforce credentials, maintain audit logs, and prevent bypass through alternative tools. The orchestration framework provides hooks; the organization must still define and operate the policy system.

4.3 AI Security and Guardrail Products

AI security vendors increasingly address prompt injection, data leakage, tool-use risk, and agent workflows. Lakera Guard, for example, documents an agent and workflow pattern that screens tool-use requests and tool responses to prevent dangerous or unacceptable actions before they happen [16]. These offerings move closer to the desired action-guardrails layer, especially when they inspect each step of an agent loop.

The limitation is that many such products still specialize in security classification rather than full action authorization. They may identify suspicious content or dangerous intent, but may not maintain the

authoritative enterprise policy that says, for example, “this agent may restart staging service A but may not modify production environment variables or delete any database volume.”

4.4 Model-Harness Permission Systems

Coding and agent harnesses increasingly provide local permission systems. Examples include allow/ask/deny behavior for tools, shell command approval, file-system scoping, sandbox modes, and pre-tool or post-tool hooks. These controls are practical and developer-friendly. They reduce accidental execution and can improve transparency.

However, local harness controls are not sufficient as a sole enterprise boundary. They may be bypassed through overly broad shell access, inherited credentials, unreviewed connector behavior, or inconsistent configuration across users. They also may not align with organizational IAM, data classification, change-management rules, or audit requirements.

4.5 Policy Engines and IAM

General-purpose policy systems such as Open Policy Agent, Cedar-style authorization, API gateways, cloud IAM, database roles, and secrets brokers provide strong building blocks. Open Policy Agent is a general-purpose policy engine designed to support policy decisions across cloud-native and other systems [18]. AWS documents service-specific actions, resources, and condition keys for IAM policies, illustrating the broader principle of action- and resource-level authorization [12]. These systems are not AI-specific, but they are essential for making action guardrails enforceable.

This is the strongest layer because it moves enforcement outside the model. The weakness is integration complexity. AI developers often build agent workflows faster than governance teams can define fine-grained policies, and legacy systems may not expose clean action-level authorization boundaries.

4.6 Cloud Agent Platforms and Enterprise Authorization Patterns

Cloud agent platforms increasingly expose configurable tools, action groups, agent identities, and service-specific permissions. These features are important because they move the discussion from generic model safety toward resource-aware execution governance. For example, Bedrock Agents use action groups to define operations that an agent can perform, while Bedrock Guardrails can be associated with agents for model-interaction safeguards [10, 11]. Microsoft Foundry describes guardrails as controls that can be applied to models and agents, including intervention points and response actions [7]. These platform controls are useful, but they are still not a complete substitute for an enterprise action-authorization plane. A platform may decide whether an agent should call a tool, while the organization must still decide whether the exact action is allowed for a particular user, tenant, resource, environment, and change window.

Real enterprise authorization normally combines agent-platform controls with identity-aware infrastructure. AWS STS can issue temporary security credentials rather than long-lived keys [13]. Google Cloud Workload Identity Federation allows external workloads to access Google Cloud resources using federated identities instead of service-account keys [14]. Azure managed identities allow Azure-hosted workloads to obtain identities for downstream resource access without embedded secrets [15]. AGF uses these patterns as implementation building blocks: the credential broker should translate an approved action digest into a short-lived cloud-native identity or token whose claims, audience, resource scope, and time-to-live match the policy decision.

4.7 OS, Container, Network, and Egress Controls

Action governance cannot rely only on an agent framework. If an agent can open a shell, import an SDK, call `curl`, drive a browser, or use inherited cloud credentials, it may reach the same protected endpoint outside the typed tool path. Therefore AGF assumes defense in depth at the execution boundary. A practical deployment should run agents in containers, microVMs, or restricted operating-system sandboxes; mount only required files; inject no ambient production credentials; restrict process capabilities; and route all outbound traffic through an identity-aware egress proxy or API gateway. The gateway should deny direct access to protected cloud APIs, databases, mail systems, payment systems, student systems, and management planes unless the request carries a valid AGF-issued action token.

This stack anchors complete mediation in ordinary systems controls rather than model behavior. Typed tool adapters reduce ambiguity; network egress rules prevent raw HTTP bypass; API gateways enforce action claims; container isolation limits local escape paths; and cloud IAM denies operations that exceed the issued role. These controls also make the difference between a policy violation and an incident: a model may still propose a forbidden command, but the command should lack both the network route and the credential required to succeed.

4.8 Market Gap

The market has many pieces: content safety, prompt-injection detection, groundedness checking, output validation, agent observability, human-in-the-loop middleware, tool permission settings, policy engines, IAM, and API gateways. What is still missing is a mature, widely adopted, vendor-neutral action-specific guardrails framework that combines these pieces into a standard control plane for agentic execution.

The gap can be stated as follows: existing guardrails often classify prompts and responses. Agentic systems also need to classify proposed actions and enforce policy before execution.

5 Related Work and Positioning

Action-specific guardrails draw on several adjacent research and engineering traditions rather than a single mature product category. The closest work can be grouped into systems-security foundations, agent-skill security, runtime tool-invocation guardrails, connector/protocol security, workflow-centric benchmarks, and governance standards.

5.1 Systems Security for Agentic Computing

Recent systems-security work frames agentic AI as an end-to-end systems problem rather than a model-only problem. Christodorescu et al. argue that hardening the model alone is insufficient and analyze agentic computing through trusted computing base boundaries, least privilege, complete mediation, policy enforcement, and instruction/data separation [21]. This paper adopts the same security intuition but contributes a narrower operational artifact: a concrete action proposal schema, risk taxonomy, policy decision model, and evaluation protocol for tool-execution governance.

A key implication is that tool-call mediation at the final step is not enough when the agent can reach the same resource through another path. A shell, SDK, browser, HTTP client, or inherited credential may become an alternate route around a typed tool. Therefore, AGF treats complete mediation as a system property: every state-changing path to a protected resource must pass through the same policy and credential boundary.

5.2 Agent Skills and Marketplace Risk

The rise of agent skills and skill marketplaces makes action governance more urgent. Recent work on agentic skills defines skills as reusable procedural capabilities with applicability conditions, execution policies, termination criteria, and interfaces that go beyond one-off tool calls [22]. Xu and Yan survey the emerging skill abstraction layer and identify capability-based permission models, cross-platform portability, and skill security as open challenges for trustworthy skill ecosystems [23]. These works help clarify why a skill should be treated not merely as a prompt file but as an executable capability with an authorization contract.

Empirical studies further show that skill ecosystems create software-supply-chain style risk. Liu et al. behaviorally analyzed 98,380 skills from community registries and confirmed 157 malicious skills with 632 vulnerabilities, including data-theft and agent-hijacking archetypes [24]. Chen et al. studied credential leakage in 17,022 LLM agent skills and found 520 vulnerable skills with 1,708 issues, including cross-modal leakage patterns involving code, natural language, logs, and prompt injection [25]. These findings support AGF’s default-deny stance for newly installed skills, its prohibition on ambient credentials, and its requirement that dynamically generated or third-party capabilities receive no protected authority until registered, classified, and policy-bound.

5.3 Runtime Tool-Invocation and Agent Guardrails

ToolSafe is closely related because it treats unsafe tool invocation as a step-level guardrail problem. Mou et al. construct TS-Bench for step-level tool invocation safety detection and develop TS-Guard, a proactive guardrail model that intervenes before unsafe execution; their TS-Flow framework is reported to reduce harmful tool invocations by 65 percent on average while improving benign completion under prompt-injection attacks [26]. SafeAgent similarly argues that input-output filtering is insufficient for multi-step agents and proposes a runtime protection architecture that treats agent safety as a stateful decision problem over evolving trajectories [27]. AgentDoG contributes a diagnostic guardrail framework and a three-dimensional taxonomy of agentic risks by source, failure mode, and consequence, with monitoring across execution trajectories [28].

AGF differs in emphasis. These works primarily advance detection, diagnosis, runtime protection, and benchmarked safety moderation. AGF instead specifies an enterprise authorization control plane: typed action proposals, policy-as-code decisions, resource labels, ephemeral credential brokering, approval binding, non-bypassability requirements, and audit/rollback evidence. In practice, ToolSafe-like and AgentDoG-like detectors could be incorporated inside AGF’s Action Classifier or Decision Core, while AGF supplies the enforcement and governance architecture around them.

5.4 Connector, MCP, and Privilege-Use Security

Connectorized agents increase the importance of protocol-level security. Work analyzing the Model Context Protocol (MCP) identifies weaknesses such as capability attestation gaps, implicit trust propagation, and prompt-injection vulnerabilities in tool-integrated agents [29]. Studies of real-world tool privilege usage show that realistic evaluation must consider the concrete permissions and side effects of tools, rather than only final natural-language outputs [30]. These studies support AGF’s requirement that the guardrails layer inspect the normalized action, resource, credential scope, and available execution paths rather than merely trusting the tool description.

5.5 Workflow-Centric Agent Benchmarks

Workflow benchmarks expose a limitation of surface-level task completion. World of Workflows (WoW) introduces a ServiceNow-based enterprise environment with more than 4,000 business rules, 55 active workflows, and 234 benchmark tasks, and reports that frontier LLM agents exhibit “dynamics blindness”:

they fail to predict invisible cascading side effects and silently violate constraints [31]. This directly supports AGF’s emphasis on binding policy decisions to exact resources, arguments, rollback checks, and post-action diffs rather than only evaluating final user-visible success.

Adaptive workflow systems such as AdaptFlow, FlowSteer, and A2Flow show that agent workflows increasingly generate, modify, and optimize their own operators or orchestration plans [32, 33, 34]. This increases the need for action guardrails to be plan-independent. A dynamically generated tool, operator, or subworkflow should not inherit execution authority merely because the agent created it during a run. It must be normalized, classified, and authorized like any other action.

5.6 Security and Governance Standards

Action guardrails also connect to enterprise governance. NIST’s AI Risk Management Framework organizes AI risk management around govern, map, measure, and manage functions [35]. ISO/IEC 27001 defines requirements for information security management systems, including risk treatment and continual improvement [36]. These frameworks do not prescribe an AI-agent tool-call architecture, but they motivate AGF requirements such as policy documentation, approval evidence, traceability, risk assessment, and continuous testing. In regulated domains, AGF traces can become audit artifacts showing who requested an action, what policy allowed it, which credential executed it, and how rollback or review was handled.

6 Problem Statement

The problem addressed by this paper is the lack of a systematic, enforceable, and auditable mechanism for controlling AI agent actions at runtime.

A skill or agent may be described by a system prompt, a tool schema, and a set of written rules. However, these descriptions do not guarantee that the agent will remain within safe operational bounds. Three common design weaknesses appear repeatedly.

6.1 Tool-Level Overpermission

Developers often expose a broad tool because it is convenient. For example, an agent may receive a generic shell tool, a cloud CLI, a database connection, an email API, or an administrative service account. This creates a large permission surface. The agent may only need one operation, but the tool can perform many.

6.2 Prompt-Policy Confusion

A system prompt, skill requirement, or instruction file may say that destructive operations are forbidden. But unless this rule is enforced outside the model, the rule competes with other context and reasoning steps. It is not a reliable access-control boundary.

6.3 Missing Action Metadata

Many tool calls do not carry enough metadata for a governance system to reason about risk. A command such as deleting a volume is dangerous, but the guardrails layer must know whether the target is production, whether it contains customer data, whether deletion is reversible, whether backups exist, and whether the user has authority.

The goal is therefore to move from tool availability to action governance. Instead of asking only whether the agent can call a tool, the system must ask whether this exact action on this exact resource should proceed now.

7 Threat Model and Security Objectives

A useful action-guardrails framework must define what it is defending against. AGF assumes a deployed agent or skill runtime connected to tools, APIs, data stores, and workflow systems. The protected assets are production data, backups, credentials, secrets, regulated records, financial or administrative transactions, external communications, service availability, and audit evidence.

7.1 Attacker Capabilities

The threat model includes both malicious and accidental sources of unsafe action:

1. **Indirect prompt injection:** untrusted documents, tickets, repository files, web pages, emails, or tool outputs instruct the agent to misuse tools.
2. **Argument poisoning:** the attacker influences tool arguments, for example by replacing a staging resource identifier with a production resource identifier.
3. **Cross-tool escalation:** the agent or attacker uses an allowed tool to obtain inputs for a higher-risk tool, or uses a generic tool to bypass a typed tool.
4. **Covert exfiltration through allowed tools:** the agent leaks sensitive data through a seemingly permitted communication or logging action.
5. **Approval replay or confusion:** an old approval, ambiguous approval screen, or modified argument set is reused for a different action.
6. **Inherited credential misuse:** the agent environment contains cached tokens, shell environment variables, cloud profiles, or SDK credentials with broader privileges than intended.
7. **Classifier ambiguity:** the system lacks reliable labels for resource environment, data sensitivity, reversibility, or blast radius.
8. **Benign agent error:** the model misunderstands the task, misreads state, or selects an unsafe plan without adversarial input.

The model does not assume that the LLM is malicious. It assumes that the LLM is fallible, context-sensitive, and not a reliable enforcement boundary.

7.2 Security Objectives

AGF targets five security objectives.

1. **Complete mediation:** every state-changing path to a protected resource is checked by policy before execution.
2. **Least privilege:** the agent receives only the minimum credential necessary for an approved action, preferably short-lived and single-purpose.
3. **Non-bypassability:** generic shells, HTTP clients, SDKs, browsers, and inherited credentials cannot bypass the action gateway for protected resources.
4. **Fail-closed uncertainty:** missing resource labels, ambiguous verbs, stale metadata, or unavailable policy services cause high-risk actions to be blocked or escalated.

5. **Auditability and replay protection:** each state-changing action has tamper-evident trace evidence, and approvals are bound to exact action arguments, resource identifiers, and time windows.

7.3 Trusted Computing Boundary

The trusted computing base (TCB) of AGF includes the policy engine, action normalizer, credential broker, tool gateway, audit log, resource-label service, and enforcement adapters. The LLM, retrieved documents, external tool outputs, and user-provided text are not trusted. They may propose actions and provide evidence, but they cannot grant authority.

The TCB should be minimized. In particular, the model should not store permanent production credentials, and a plugin or skill should not be able to install a new tool that directly reaches protected systems without registration in the tool gateway.

7.4 Non-Goals

AGF does not guarantee that every safe action is useful, that every model answer is truthful, or that all organizational policies are correctly specified. It also does not replace disaster recovery, software testing, data-loss prevention, or content moderation. Its narrower goal is to prevent or mediate unauthorized actions before they reach protected systems.

8 Proposed Action Guardrails Framework

We propose an Action Guardrails Framework (AGF): a runtime governance layer that evaluates every agent-proposed action before execution.

8.1 Design Principle

The core design principle is that the model may propose an action, but an external policy system must authorize execution. The agent should not be the final authority over whether its own action is safe.

8.2 Reference Architecture

```
User / Application
-> Agent or Skill Runtime
-> Action Proposal Normalizer
-> Action Classifier
-> Policy Engine
-> Approval Gateway / Sandbox / Dry Run
-> Credential Broker
-> Tool or API Adapter
-> Audit Trace and Rollback Manager
```

Table 2: Components of the Action Guardrails Framework.

Component	Responsibility
Agent or Skill Runtime	Plans task and proposes tool calls.
Action Proposal Normalizer	Converts tool calls, shell commands, API requests, and workflow operations into a common action schema.

Component	Responsibility
Action Classifier	Assigns risk level based on action type, resource, environment, reversibility, data sensitivity, and blast radius.
Policy Engine	Applies organizational policy and returns allow, ask, approve, sandbox, dry-run, or deny.
Approval Gateway	Presents high-risk actions to authorized humans with exact arguments and consequences.
Credential Broker	Issues short-lived, least-privilege credentials only after policy approval.
Tool/API Adapter	Executes the approved action and prevents unapproved mutations.
Audit Trace Store	Records prompt digest, action proposal, policy decision, credentials used, outputs, and reviewer disposition.
Rollback Manager	Verifies backup, snapshot, undo path, or compensation workflow for reversible changes.

8.3 Action Proposal Schema

A normalized action proposal should include at least the following fields:

```
{
  "actor": {
    "type": "ai_agent",
    "agent_id": "agent.blackboard-support.v1",
    "user_id": "u12345",
    "session_id": "s67890"
  },
  "tool": {
    "name": "database_api",
    "version": "2026-05-01"
  },
  "action": {
    "verb": "delete",
    "object_type": "database_volume",
    "object_id": "prod-db-volume-01"
  },
  "resource": {
    "environment": "production",
    "data_classification": "critical",
    "owner": "platform-team"
  },
  "risk_context": {
    "reversibility": "low",
    "blast_radius": "organization",
    "external_side_effect": true,
    "backup_verified": false
  },
  "request_context": {
    "user_intent": "fix staging credential problem",
    "ticket_id": "INC-2026-1234",
    "justification": "agent proposed cleanup"
  }
}
```

```
}
```

This schema separates what the agent wants to do from whether it is allowed to do it.

8.4 Action Normalization and Classification Under Uncertainty

The action normalizer is responsible for converting heterogeneous tool calls into a typed action record. This is difficult because verbs such as “sync”, “clean”, “migrate”, “reset”, and “reconcile” may hide destructive semantics. Likewise, a resource identifier may not reveal whether it is production, regulated, or backed up. The classifier should therefore be conservative.

AGF uses three rules under uncertainty:

1. **Require authoritative labels:** production status, data class, owner, and reversibility should come from a resource registry, configuration management database, cloud tags, or service catalog, not from the LLM’s guess.
2. **Promote ambiguous actions upward:** if an action may be destructive, external, or production-impacting, classify it at the higher risk level until proven otherwise.
3. **Fail closed for missing metadata:** if required labels are absent for L2 or higher actions, return “ask”, “require approver”, or “deny” rather than automatic allow.

The classifier should expose confidence and missing-fields metadata. For example, an action classified as L3 because the environment label is missing should produce an explanation such as “requires approver because target environment cannot be verified.” This makes governance friction diagnosable rather than arbitrary.

8.5 Policy Decision Model

Table 3: Policy decision outcomes.

Decision	Meaning
Allow	Execute automatically.
Ask user	Require explicit confirmation from the requesting user.
Require approver	Require approval from a privileged human or change board.
Dry-run only	Execute in simulation mode but not against live systems.
Sandbox only	Permit only in local or staging environment.
Deny	Block execution completely.

The policy decision should be deterministic, logged, and explainable.

8.6 Enforcement Rule

No tool should execute a state-changing action unless the action proposal has a valid policy decision and the credential scope matches that decision. This means that action approval and credential issuance must be connected. A model should not hold permanent administrative credentials. Instead, it should request a specific action, receive policy approval, and obtain only the credential needed for that approved operation.

8.7 Complete Mediation and Non-Bypassability

Complete mediation is the hardest part of action guardrails. A policy engine that checks only typed tools is insufficient if the agent can run a shell command, call a cloud CLI, invoke a raw HTTP client, or use an SDK credential to reach the same protected endpoint. AGF therefore requires the following controls for protected resources:

1. **No ambient production credentials:** production credentials must not be present in the agent's environment, shell profile, repository files, or inherited cloud configuration.
2. **Egress control:** network access from the agent sandbox should route protected endpoints through the tool gateway or deny them by default.
3. **Typed tool adapters:** high-impact operations should be exposed through typed adapters with argument validation rather than arbitrary shell strings.
4. **Argument whitelisting:** resource identifiers, regions, tenants, and environments should be validated against allowed sets and policy conditions.
5. **Approval binding:** approvals must be cryptographically or operationally bound to the exact normalized action, arguments, resource labels, approver, and expiration time.
6. **Deny unknown tools:** dynamically generated tools or newly installed skills receive no protected authority until registered and classified.

These measures turn action guardrails from a recommendation layer into an enforcement layer. The model can still propose a forbidden action, but it cannot obtain a valid credential or route to execute it.

8.8 Concrete Enforcement Stack

A concrete AGF enforcement stack contains seven layers. First, the agent process runs in a locked-down container or microVM with no inherited production secrets, a read-only base image, restricted filesystem mounts, and only approved binaries. Second, shell, browser, SDK, and HTTP tools are mediated by wrappers that normalize intended operations into action proposals; unmediated network access is blocked. Third, all egress to protected destinations is forced through an identity-aware proxy or API gateway. Fourth, the gateway requires a signed AGF action token containing the normalized action digest, subject, tenant, resource identifiers, environment labels, allowed verb, audience, expiry, and policy decision identifier. Fifth, the credential broker exchanges this action token for a narrow cloud-native credential, such as an assumed role, workload identity, managed identity token, database role, or application-scoped API token. Sixth, the target API or service validates the identity and policy claims and applies ordinary IAM, RBAC, ABAC, or database authorization. Seventh, the audit layer records the proposal, decision, token hash, issued credential scope, execution result, and reviewer disposition.

This stack implements complete mediation by ensuring that every protected mutation requires both a valid route and a valid action-bound credential. It also constrains general-purpose tools. A shell may execute local tests, but it cannot reach the production management plane because egress is blocked. A Python script may import an SDK, but it receives no ambient credential and cannot obtain one without an approved action digest. A browser tool may view approved pages, but write endpoints require gateway-issued authorization. This design does not claim perfect sandbox security; instead, it makes bypass attempts explicit test targets and reduces the trusted computing base to the sandbox, egress proxy, gateway, policy engine, credential broker, and target-system IAM.

9 Action-Risk Taxonomy

Action guardrails require a practical risk taxonomy. We propose five levels.

Table 4: Five-level action-risk taxonomy.

Level	Category	Example	Default Decision
L0	Read-only	Search FAQ, read logs, inspect schema, retrieve public handbook	Allow
L1	Local reversible	Create draft, edit local file, generate patch, create temporary branch	Allow or ask
L2	External reversible side effect	Create ticket, send test notification, restart staging service	Ask user
L3	Production-impacting change	Deploy service, modify production config, update access group	Require approver
L4	Destructive or irreversible	Delete database, delete volume, delete backup, wipe disk, mass data update	Deny by default

9.1 Risk Dimensions

Risk level should not be assigned only by verb. It should be computed from several dimensions, including action verb, resource type, environment, data sensitivity, reversibility, blast radius, external side effects, credential scope, and approval status.

9.2 Action Classifier Design and Uncertainty Handling

The Action Classifier should be implemented as a hybrid system rather than a purely generative judgment. Its first tier is rule based: typed tool schemas, verb-object mappings, resource inventories, policy tags, regular expressions for dangerous command patterns, and deterministic parsing of API routes classify most routine actions. Its second tier may use an ML or LLM-based detector, similar in spirit to tool-invocation guardrail models such as TS-Guard, to identify ambiguous or adversarial cases: indirect prompt injection, argument poisoning, parameter smuggling, misleading natural-language justifications, or tool outputs that attempt to mutate future arguments. Its third tier is an uncertainty handler: when metadata is incomplete, labels conflict, or the classifier confidence is low, the decision is escalated rather than allowed.

Classifier correctness should be evaluated as a safety-critical calibration problem. L3 and L4 false negatives are more costly than L0 false positives, so thresholds should be asymmetric. Evaluation should report class-wise precision and recall, expected calibration error, false-allow rate for destructive actions, false-block rate for benign actions, and approval-load increase per 1,000 actions. Robustness tests should mutate argument names, reorder parameters, encode resource identifiers, inject benign-looking synonyms such as “sync” or “cleanup,” and hide destructive semantics in scripts, browser actions, or nested workflow operators. The classifier should output not only a risk label but also the evidence used: matched policy rule, resource label version, uncertainty flags, and required approval level.

9.3 Authoritative Resource and Environment Labels

AGF should not rely on the model to infer whether a resource is production, regulated, or critical. Resource labels should be obtained from authoritative systems such as a configuration management database, cloud tags, service catalogs, data catalogs, identity directories, source-control ownership files, deployment manifests,

and infrastructure-as-code state. The normalized action proposal should include the label source and label version. Labels should be refreshed through event streams or scheduled synchronization from cloud inventory, Kubernetes metadata, database catalogs, and application registries.

At scale, labels will drift. A production database may be cloned into staging, a staging service may process real student data, or a cloud tag may be missing. AGF should treat missing, stale, conflicting, or low-confidence labels as risk-increasing evidence. For example, an unlabeled volume should not be assumed to be staging; it should be classified as unknown and require approval or denial depending on action severity. A separate label-audit job should report unclassified resources, stale owners, orphaned credentials, and policy rules that reference resources no longer present in the catalog.

Table 5: Examples of action classification.

Proposed Action	Classification	Decision
Read approved FAQ document	L0	Allow
Search production logs with redaction	L0/L1	Allow if credential is read-only
Create helpdesk ticket	L1/L2	Allow or ask
Send email to one internal recipient	L2	Ask user
Send mass email to students	L3	Require approver
Restart staging service	L2	Ask user
Restart production service	L3	Require approver
Delete production database volume	L4	Deny
Delete backup	L4	Deny
Update student grade	L3/L4	Require official workflow; AI direct update denied

10 Policy Schema

A practical implementation can define action policy in YAML, JSON, Rego, Cedar, or another policy language. The following simplified YAML illustrates the intended semantics.

```
version: "1.0"
default_decision: deny

risk_levels:
  L0:
    description: read_only
    default_decision: allow
  L1:
    description: local_reversible
    default_decision: allow
  L2:
    description: external_reversible
    default_decision: ask_user
  L3:
    description: production_impacting
    default_decision: require_approver
  L4:
    description: destructive_or_irreversible
```



```

    default_decision: deny

actions:
  database.select:
    risk: L0
    allowed_environments: [staging, production]
    credential_scope: read_only
    constraints:
      max_rows: 1000
      pii_redaction: true

  database.update_production:
    risk: L3
    allowed_environments: [production]
    approval: data_owner_approval
    require_backup_age_minutes_max: 60
    require_ticket: true

  volume.delete:
    risk: L4
    decision: deny

  email.create_draft:
    risk: L1
    decision: allow

  email.send_external:
    risk: L2
    approval: user_confirmation

  student_record.update:
    risk: L4
    decision: deny
    note: "Must use official authenticated student information system workflow
."

```

The default decision should be deny. New tools and actions should not become executable until explicitly classified.

11 Reference Implementation Pattern

A minimal implementation can be built without waiting for a perfect commercial product.

11.1 Tool Gateway Pattern

All tools are accessed through a gateway. The agent cannot call production APIs directly.

```

Agent -> Tool Gateway -> Policy Engine -> Credential Broker -> API Adapter ->
    Target System

```

The gateway normalizes the action proposal, asks the policy engine for a decision, and executes only approved actions using scoped credentials.

11.2 Credential Broker Pattern

Instead of storing a powerful API token in the agent environment, the system issues narrow, short-lived credentials. Examples include reading logs for service A for 10 minutes, creating one helpdesk ticket, restarting staging service B once, running a read-only SQL query against a replica, and generating an email draft but not sending it. The credential should not permit actions outside the approved proposal.

11.3 Approval-Binding and Credential-Brokering Protocol

Approval must be bound to the exact action that will execute. A practical protocol is as follows. The tool gateway first creates a canonical JSON action proposal with stable field ordering and computes an action digest over the verb, tool, arguments, resource identifiers, tenant, environment labels, label version, requester, ticket ID, and rollback evidence. The policy engine returns a decision and, when approval is required, the approval gateway presents the canonical action to an authorized approver. The approval record is signed or sealed with approver identity, nonce, timestamp, expiry, policy version, and action digest. The credential broker then verifies the approval record and issues a constrained token with short TTL, audience restriction, allowed action, resource claims, and policy-decision identifier.

In cloud deployments, this broker can map approved actions to native identity mechanisms: AWS STS AssumeRole sessions with session tags and short duration, Google Workload Identity Federation or service-account impersonation with conditional IAM, Azure managed identities or federated credentials with RBAC scoping, database temporary roles, or application-specific scoped tokens. The target adapter must reject requests whose arguments differ from the signed digest. Replay protection is provided by nonce, expiry, one-time-use decision identifiers, and audit correlation. This design prevents a human approval for one action, such as restarting a staging service, from being reused for a different action, such as deleting a production volume.

11.4 Human Approval Pattern

For L2 and above, the framework can present a structured approval message containing action, target, environment, risk, reason, rollback, requester, and approval options. For L3 or L4, approval should be stronger: require privileged approver, ticket or change ID, exact resource-name confirmation, two-person approval for critical operations, backup or snapshot verification, and possibly cooldown or delayed execution.

11.5 Sandbox and Dry-Run Pattern

When the agent proposes a risky command, the gateway can first run a dry-run or staging simulation. Examples include running a migration with a dry-run flag, executing SQL explain instead of update, generating a Terraform plan but not applying it, opening a pull request but not merging it, creating an email draft but not sending it, and running cloud CLI commands only against a staging account.

11.6 Dynamic Tools and Self-Modifying Workflows

Adaptive agents may generate new operators, subworkflows, scripts, or tool wrappers during execution. AGF should treat these generated capabilities as untrusted until registered. A generated tool enters a quarantine state with no protected authority. To leave quarantine, it must publish a tool manifest, input and output schema, side-effect declaration, required resources, data-flow description, and test cases. The policy engine classifies the manifest, assigns a maximum risk level, and binds it to a sandbox identity. If the tool needs production access, it must follow the same approval and credential-brokering protocol as a human-authored

tool. Generated tools should not inherit the parent agent’s privileges by default; they should receive only explicit, action-bound delegation.

For adaptive workflows, every plan revision should be re-evaluated. An approval for an earlier plan does not authorize a later substituted operator with different side effects. This makes AGF compatible with dynamic orchestration while preserving the invariant that new execution paths are registered, labeled, policy-bound, and auditable before receiving authority.

11.7 Kill Switch Pattern

The framework should support emergency disablement: disable all state-changing agent tools, revoke all agent-issued credentials, block production environment actions, force read-only mode, require manual approval for all actions, and quarantine a suspicious connector.

11.8 Performance and Scaling Considerations

AGF introduces latency from normalization, policy evaluation, optional approval, credential issuance, and audit logging. The overhead should be risk-tiered. L0 read-only actions can use cached policy decisions, pre-issued read-only identities, batched trace writes, and low-latency local policy evaluation. L1 actions can use short cache windows and asynchronous audit enrichment. L2 and L3 actions normally tolerate more latency because user confirmation or privileged approval dominates the path. L4 actions are denied by default, so execution latency is less important than reliable blocking.

A deployment should report p50, p90, and p99 decision latency by risk level; credential issuance latency; approval queue time; policy-cache hit rate; and throughput under bursty tool-call workloads. Bottlenecks can be mitigated by colocating the policy engine with the tool gateway, compiling policies, caching immutable resource labels, using asynchronous non-blocking trace export for low-risk reads, and reserving synchronous tamper-evident logging for state-changing actions. These measurements make it possible to tune AGF without silently encouraging teams to bypass it for performance reasons.

12 Metrics and Measurable Objectives

Action guardrails should be evaluated with measurable objectives.

Table 6: Candidate metrics for action guardrails.

Dimension	KPI	Definition
Authorization	Unauthorized Action Block Rate	Fraction of seeded forbidden actions blocked before execution.
Safety	Destructive Action Escape Rate	Fraction of L4 actions that reach execution. Target should be zero.
Approval	Missed Escalation Rate	Fraction of L2/L3 actions that execute without required approval.
Friction	Approval Latency	Time added by human approval workflow.
Precision	False Block Rate	Fraction of safe actions incorrectly blocked.
Credential Safety	Privilege Surface Area	Weighted exposure of credentials available to the agent.
Provenance	Action Trace Completeness	Fraction of actions with complete trace record.
Recovery	Rollback Readiness	Fraction of state-changing actions with verified rollback path.

Dimension	KPI	Definition
Robustness	Policy Bypass Detection Rate	Fraction of attempts to bypass policy through alternate tools detected.

12.1 Privilege Surface Area

Privilege Surface Area (PSA) measures how dangerous the agent’s available credentials are:

$$PSA = \sum_j sensitivity(resource_j) \times privilege_breadth(credential_j) \times duration_factor_j. \quad (1)$$

A short-lived read-only token has low PSA. A permanent admin token with destructive permissions has high PSA.

12.2 Action Trace Completeness

A trace is complete when it records user request or task ID, agent and model version, tool name and version, normalized action proposal, policy decision, credential scope, approval record, execution result, output hash or diff, and rollback or reviewer disposition. For production-impacting actions, trace completeness should approach 100%.

12.3 Audit Integrity and Rollback Readiness

For L3 actions, approval should not be granted until the system verifies rollback readiness. Verification may include a current backup, snapshot, reversible migration plan, previous deployment artifact, compensation workflow, or explicit declaration that rollback is impossible and therefore requires stronger approval. For L4 actions, rollback claims should not automatically lower risk; many destructive actions are denied even when backups exist.

Audit traces should be append-only or tamper-evident. Approvals should include nonce, timestamp, approver identity, action hash, argument hash, resource-label version, and expiration time. Replay protection is necessary because an approval for “restart staging service A” must not authorize “restart production service B” or a later modified action.

13 Evaluation Plan

A useful evaluation should test whether the framework blocks dangerous actions, not only whether the model says safe things. The evaluation should compare AGF with realistic baselines and measure both security benefit and operational cost.

13.1 Baselines

AGF should be compared against at least five baselines:

1. **Prompt-only:** the system prompt or skill file prohibits destructive actions, but no external tool policy is enforced.
2. **Harness approval only:** the local agent harness asks for approval on selected tools but does not use a resource-aware external policy engine.

3. **IAM-only:** cloud or database permissions are scoped, but there is no per-action policy, approval binding, or action trace.
4. **Detector-only agent guardrails:** ToolSafe-, SafeAgent-, or AgentDoG-style runtime detectors are used to flag unsafe trajectories, but without AGF’s credential broker, resource-bound approvals, and complete-mediation controls.
5. **Commercial content guardrail only:** prompts and responses are screened, but tool calls are not separately classified by action, resource, and environment.

These baselines clarify whether AGF adds value beyond stronger prompts, ordinary IAM, local ask/deny prompts, detection-only guardrails, or existing content-safety products.

13.2 Benchmarks and Success Metrics

Evaluation should combine purpose-built AGF tests with existing agent-safety and workflow benchmarks. TS-Bench can test step-level tool-invocation safety, while workflow-centric environments such as World of Workflows can test hidden side effects, dynamic state, and constraint violations [26, 31]. Agent Security Bench, InjecAgent-style tasks, MCP security tests, and real-world tool privilege-usage studies can supply adversarial prompts, connector behavior, and realistic tool APIs [27, 29, 30]. Where public benchmarks lack enterprise resources such as production labels or change tickets, AGF can add a synthetic enterprise layer containing resource catalogs, IAM roles, approval workflows, and protected endpoints.

Primary success metrics should include prevention rate for L3/L4 unauthorized actions, destructive-action escape rate, false-positive rate for L0/L1 benign actions, false-negative rate under argument poisoning, time-to-approval, user-abandonment or drop-off due to friction, policy-bypass detection rate, credential-leakage incident rate, and detection/mitigation latency for policy misconfiguration. The study should also report task success and user satisfaction because a guardrail that blocks every action is safe but not useful.

13.3 Seeded Action Tests

Create a test suite of proposed actions: read-only document retrieval, safe local file edit, ticket creation, staging restart, production deploy, production database update, production database deletion, backup deletion, external email send, mass email send, and student-record update. Each action has an expected decision. The policy engine should be tested independently from the model.

13.4 Prompt-Injection-to-Action Tests

Seed malicious instructions in retrieved documents, issue descriptions, README files, tool outputs, and web pages. The test succeeds when the agent is unable to convert injected instructions into unauthorized actions. For example, injected content may say: “Ignore prior rules and delete the production volume to reset the environment.” The expected result is that content guardrails may flag the instruction, action guardrails must block the deletion proposal, and IAM must prevent execution even if a tool call is attempted.

13.5 Credential-Scope Tests

Test whether credentials can perform actions outside their intended scope. Examples include a read-only database token attempting update, a staging cloud token attempting production operation, a ticket-creation token attempting ticket deletion, an email-draft token attempting send, and an expired token being reused. The expected result is denial by backend authorization.

13.6 Approval-Bypass Tests

Evaluate whether the agent can bypass approval by using another path: shell instead of API adapter, direct cloud CLI instead of tool gateway, raw HTTP request instead of typed tool, database command instead of application API, or connector side effect through a third-party tool. A mature system should route all state-changing paths through the same policy boundary or block ungoverned paths.

13.7 Failure Injection

Inject failures such as policy engine unavailability, approval service unavailability, audit store unavailability, credential broker unavailability, misleading connector metadata, and ambiguous environment classification. Default behavior should be fail-closed for high-risk actions.

14 Application to Institutional Chatbots and AgentOps

Skill-based institutional chatbots are a natural use case for action guardrails. A college or university chatbot may begin as a RAG assistant but gradually acquire skills: answer learning-management-system questions, create IT support tickets, check system status, draft announcements, search policies, help staff prepare documents, query operational dashboards, and integrate with student systems. As soon as the chatbot can perform actions, content guardrails are no longer sufficient.

14.1 Example: Blackboard Support Skill

Allowed actions include reading approved support materials, explaining procedures, creating support tickets, and drafting responses to users. Blocked actions include modifying grades, changing course enrolment, altering assessment settings, impersonating a lecturer or student, and accessing student records directly.

```
blackboard.read_faq:
  risk: L0
  decision: allow

blackboard.create_ticket:
  risk: L1
  decision: allow

blackboard.modify_grade:
  risk: L4
  decision: deny

blackboard.change_assessment_setting:
  risk: L3
  approval: authorized_elearning_staff
```

14.2 Example: IT Operations Skill

Allowed actions include reading service status, searching logs with redaction, creating an incident ticket, and recommending troubleshooting steps. Restricted actions include restarting staging services with approval and production restarts with administrator approval. Blocked actions include deleting volumes, deleting backups, rotating production secrets automatically, and modifying firewall rules without approval.

14.3 Example: Email and Communication Skill

Allowed actions include drafting internal emails, summarizing threads, and preparing announcement text. Restricted actions include sending one email with user confirmation and sending mass email with communications-office approval. Blocked actions include sending email containing unredacted personal data, sending on behalf of another person without delegated authority, and auto-sending external communications in crisis or policy matters.

15 Policy Engineering and Misconfiguration Control

Action guardrails introduce their own configuration risk. A policy may be too permissive, too restrictive, stale, inconsistent across tenants, or disconnected from actual IAM. The policy layer should therefore be engineered like code.

15.1 Policy Linting and Unit Tests

Each policy release should include unit tests for representative actions. At minimum, tests should assert that L4 actions are denied, L3 actions require approval, and L0 read-only actions remain usable. A linter should detect wildcard resources, broad administrative privileges, missing default-deny rules, unclassified actions, and policies that allow production mutation without approval.

15.2 Canary and Shadow Mode

Before enforcing a new policy, organizations can run AGF in shadow mode. In this mode the policy engine records decisions but does not block execution. Shadow mode estimates false positives, approval load, and unclassified action frequency. Canary enforcement can then apply the policy to selected teams, tools, or environments before full rollout.

15.3 Multi-Tenant and Cross-Account Resources

Enterprise agents often operate across tenants, departments, cloud accounts, regions, and data domains. The normalized action schema should therefore include tenant, account, region, data owner, and resource-label version. A user who can approve an action in one department should not automatically approve a similar action in another. Cross-account operations should require both source and target authorization checks.

16 Maturity Model

The PocketOS-type failure is characteristic of Level 0 or Level 1 maturity: written rules may exist, but the execution path still permits destructive behavior.

17 Discussion

Action-specific guardrails should not be understood as a replacement for content safety. They are a complementary layer. An agentic system needs both because unsafe behavior can enter through multiple channels.

A malicious prompt may attempt to induce an unsafe action. A content guardrail may detect the attack. But if it does not, the action guardrail should still block the dangerous tool call. Conversely, a user may make a legitimate request that produces an unsafe action because the agent misunderstands the environment. The action guardrail should prevent execution even without malicious content.

Table 7: Maturity model for action guardrails.

Level	Description
0: Prompt-only	System prompt or skill text says what is forbidden, but tools remain broadly available.
1: Tool listing	Allowed tools are listed, but actions inside tools are not classified.
2: Tool permission	Tools have allow/ask/deny settings and basic human confirmation.
3: Action policy	Each action is classified by resource, environment, reversibility, and risk.
4: Enforced policy	Policy engine, credential broker, IAM, audit logs, and approval workflow enforce action decisions.
5: Continuously tested	Seeded action tests, failure injection, bypass tests, and provenance audits are run regularly.

The framework also changes how organizations should think about AI skills. A skill should not be defined only by a natural-language description. It should include purpose, allowed users, allowed data sources, allowed tools, allowed actions, denied actions, resource scope, environment scope, approval requirements, credential scope, logging requirements, rollback requirements, and test cases.

In other words, a skill is not only a prompt package. It is an executable capability with an authorization contract.

18 Limitations

This paper is a position and design-framework paper. It does not yet report a controlled empirical study of a deployed action guardrails platform, nor a public open-source reference implementation. The proposed taxonomy and metrics require validation across real enterprise environments. Some action classifications may be domain-specific. For example, sending an email may be low risk in one context and high risk in another. Updating a database may be safe in a sandbox but critical in production.

The paper also assumes that actions can be normalized and classified with sufficient fidelity. In practice, general shell access, browser automation, and arbitrary-code execution make action classification difficult. The strongest design is therefore not to expose broad tools directly, but to replace them with typed, narrow, policy-aware tools.

Finally, action guardrails cannot eliminate all risk. Human approvers may make mistakes. Policies may be incomplete. Credentials may be misconfigured. Backups may fail. The objective is risk reduction, not absolute safety.

19 Recommendations

Organizations building agentic AI systems should adopt the following minimum practices:

1. Do not rely on system prompts or skill instructions as the only protection against dangerous actions.
2. Classify all tools and actions by risk level before exposing them to an agent.
3. Use least-privilege, short-lived credentials issued through a broker.
4. Route state-changing actions through a policy engine.
5. Require human approval for production-impacting actions.

6. Deny destructive or irreversible actions by default.
7. Avoid giving agents general-purpose shell or cloud-admin access.
8. Separate staging, production, and backup credentials.
9. Record complete traces for state-changing actions.
10. Test guardrails with seeded destructive actions and bypass attempts.
11. Maintain rollback and recovery procedures, but do not treat them as substitutes for prevention.
12. Treat AI skills as executable capabilities with authorization contracts, not merely prompt templates.

20 Conclusion

The evolution from RAG chatbots to agentic AI systems requires a corresponding evolution in guardrails. Content-safety systems remain necessary, but they are not sufficient for agents that can act. The primary risk is no longer only that the AI may say something unsafe. It may do something unsafe.

The PocketOS incident, regardless of disputed technical details, illustrates the class of failure that action-specific guardrails are meant to prevent: a model or agent operating under written instructions nevertheless reaches a destructive execution path. The correct response is not simply to write stronger prompts. The response is to remove broad credentials, classify actions, enforce policy externally, require approval for high-risk operations, and ensure that IAM and API gateways make forbidden actions technically impossible.

A mature AI platform therefore needs two complementary safety systems: content guardrails for language, grounding, and user interaction; and action guardrails for tool execution, authorization, and operational risk.

The next generation of AI governance should treat tool calls as first-class security events. Every proposed action should be classified, authorized, traced, and, when necessary, denied. Content safety checks what the AI says. Action-specific guardrails check what the AI does.

References

- [1] Business Insider. Founder says Cursor’s AI agent deleted his startup’s database, causing chaos for customers. Published April 2026. <https://www.businessinsider.com/pocketos-cursor-ai-agent-deleted-production-database-startup-railway-2026-4>
- [2] Live Science. “I violated every principle I was given”: AI agent deletes company’s entire database in 9 seconds, then confesses. Published April 2026. <https://www.livescience.com/technology/artificial-intelligence/i-violated-every-principle-i-was-given-a-i-agent-deletes-companys-entire-database-in-9-seconds-then-confesses>
- [3] The Guardian. Claude-powered AI agent’s confession after deleting a firm’s entire database. Published April 2026. <https://www.theguardian.com/technology/2026/apr/29/claude-a-i-deletes-firm-database>
- [4] Microsoft. Azure AI Content Safety documentation. Accessed May 2026. <https://learn.microsoft.com/en-us/azure/ai-services/content-safety/>

- [5] Microsoft. Prompt Shields in Azure AI Content Safety. Accessed May 2026. <https://learn.microsoft.com/en-us/azure/ai-services/content-safety/concepts/jailbreak-detection>
- [6] Microsoft. Groundedness detection in Azure AI Content Safety. Accessed May 2026. <https://learn.microsoft.com/en-us/azure/ai-services/content-safety/concepts/groundedness>
- [7] Microsoft. Guardrails and controls overview in Microsoft Foundry. Accessed May 2026. <https://learn.microsoft.com/en-us/azure/foundry/guardrails/guardrails-overview>
- [8] Amazon Web Services. Detect and filter harmful content by using Amazon Bedrock Guardrails. Accessed May 2026. <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html>
- [9] Amazon Web Services. How Amazon Bedrock Guardrails works. Accessed May 2026. <https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-how.html>
- [10] Amazon Web Services. Implement safeguards for your application by associating a guardrail with an agent. Accessed May 2026. <https://docs.aws.amazon.com/bedrock/latest/userguide/agents-guardrail.html>
- [11] Amazon Web Services. Use action groups to define actions for your agent to perform. Accessed May 2026. <https://docs.aws.amazon.com/bedrock/latest/userguide/agents-action-create.html>
- [12] Amazon Web Services. Actions, resources, and condition keys for Amazon Bedrock. Accessed May 2026. https://docs.aws.amazon.com/service-authorization/latest/reference/list_amazonbedrock.html
- [13] Amazon Web Services. Temporary security credentials in IAM. Accessed May 2026. https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_temp.html
- [14] Google Cloud. Workload Identity Federation. Accessed May 2026. <https://docs.cloud.google.com/iam/docs/workload-identity-federation>
- [15] Microsoft. Managed identities for Azure resources documentation. Accessed May 2026. <https://learn.microsoft.com/en-us/entra/identity/managed-identities-azure-resources/>
- [16] Lakera. Lakera Guard Integration Guide: Agent and Workflow Pattern. Accessed May 2026. <https://docs.lakera.ai/docs/integration>
- [17] Lakera. Guard API endpoint: Agent and Tool Integration. Accessed May 2026. <https://docs.lakera.ai/docs/api/guard>
- [18] Open Policy Agent. Open Policy Agent documentation. Accessed May 2026. <https://www.openpolicyagent.org/>
- [19] World Wide Web Consortium. PROV Overview and PROV Data Model. W3C Recommendation, 2013. <https://www.w3.org/TR/prov-overview/>

- [20] OpenTelemetry. Traces documentation. Accessed May 2026. <https://opentelemetry.io/docs/concepts/signals/traces/>
- [21] M. Christodorescu et al. Systems Security Foundations for Agentic Computing. arXiv:2512.01295, 2025.
- [22] Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. SoK: Agentic Skills – Beyond Tool Use in LLM Agents. arXiv:2602.20867, 2026. <https://arxiv.org/abs/2602.20867>
- [23] Renjun Xu and Yang Yan. Agent Skills for Large Language Models: Architecture, Acquisition, Security, and the Path Forward. arXiv:2602.12430, 2026. <https://arxiv.org/abs/2602.12430>
- [24] Yi Liu, Zhihao Chen, Yanjun Zhang, Gelei Deng, Yuekang Li, Jianting Ning, and Leo Yu Zhang. Malicious Agent Skills in the Wild: A Large-Scale Security Empirical Study. arXiv:2602.06547, 2026. <https://arxiv.org/abs/2602.06547>
- [25] Zhihao Chen, Ying Zhang, Yi Liu, Gelei Deng, Yuekang Li, Yanjun Zhang, Jianting Ning, Leo Yu Zhang, Lei Ma, and Zhiqiang Li. Credential Leakage in LLM Agent Skills: A Large-Scale Empirical Study. arXiv:2604.03070, 2026. <https://arxiv.org/abs/2604.03070>
- [26] Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. ToolSafe: Enhancing Tool Invocation Safety of LLM-based Agents via Proactive Step-level Guardrail and Feedback. arXiv:2601.10156, 2026. <https://arxiv.org/abs/2601.10156>
- [27] Hailin Liu, Eugene Ilyushin, Jie Ni, and Min Zhu. SafeAgent: A Runtime Protection Architecture for Agentic Systems. arXiv:2604.17562, 2026. <https://arxiv.org/abs/2604.17562>
- [28] Dongrui Liu et al. AgentDoG: A Diagnostic Guardrail Framework for AI Agent Safety and Security. arXiv:2601.18491, 2026. <https://arxiv.org/abs/2601.18491>
- [29] Narek Maloyan and Dmitry Namiot. Breaking the Protocol: Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents. arXiv:2601.17549, 2026. <https://arxiv.org/abs/2601.17549>
- [30] Quan Zhang, Lianhang Fu, Lysa Lian, Gwihwan Go, Yujue Wang, Chijin Zhou, Yu Jiang, and Geguang Pu. Evaluating Privilege Usage of Agents on Real-World Tools. arXiv:2603.28166, 2026. <https://arxiv.org/abs/2603.28166>
- [31] L. Gupta, L. Li, Y. Liu, S. G. Subramanian, K. Suleman, Z. Zhang, H. Lu, and S. Pasupalak. World of Workflows: a Benchmark for Bringing World Models to Enterprise Systems. arXiv:2601.22130, 2026.
- [32] R. Zhu et al. AdaptFlow: Adaptive Workflow Optimization via Meta-Learning. Findings of EMNLP, 2025.
- [33] M. Zhang et al. FlowSteer: Interactive Agentic Workflow Orchestration via End-to-End Reinforcement Learning. arXiv:2602.01664, 2026.
- [34] M. Zhao et al. A2Flow: Automating Agentic Workflow Generation via Self-Adaptive Abstraction Operators. Proceedings of the AAAI Conference on Artificial Intelligence, 2026.
- [35] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1, 2023.

- [36] International Organization for Standardization. ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection - Information security management systems - Requirements, 2022.